

Homework2 : 3D Scene Reconstruction

112550032 李尹瑄 (a notion link to this report: https://www.notion.so/Homework2-3D-Scene-Reconstruction-33c66320375180e384f9e9119c3ccdbf?source=copy_link)

Implementation

Code

This assignment aims to reconstruct a 3D scene by representing each point in space as a point cloud, where all points are expressed under a unified coordinate system, namely the world coordinate system. The following describes the overall pipeline in detail:

1.

Use the camera intrinsic parameters and the depth image to obtain the point cloud of each frame in the corresponding camera coordinate system, i.e., the (x,y,z) coordinates of each point with respect to that frame's camera.

```

21 def depth_image_to_point_cloud(rgb_image, depth_image):
22
23     """
24     TASK 1: Geometric Unprojection [cite: 12, 25-27]
25     Convert depth pixels (u, v, d) into 3D world points (x, y, z).
26     """
27     # 1. Convert inputs to numpy arrays
28     # 2. Convert depth to meters (Habitat depth is often scaled or normalized)
29     rgb = np.asarray(rgb_image).astype(np.float32) / 255.0
30     depth = np.asarray(depth_image).astype(np.float32)
31     depth_m = depth / 255.0 * 10.0
32
33     # 3. Create a coordinate grid for (u, v) pixels
34     h, w = depth.shape
35     u, v = np.meshgrid(np.arange(w), np.arange(h))
36
37     # Implement unprojection logic here
38     z = depth_m
39     x = (u - CX) * z / FX
40     y = -(v - CY) * z / FY
41     points = np.stack((x, y, -z), axis=-1).reshape(-1, 3)
42     colors = rgb.reshape(-1, 3)
43
44     valid = z.flatten() > 0
45     points = points[valid]
46     colors = colors[valid]
47
48     pcd = o3d.geometry.PointCloud()
49     pcd.points = o3d.utility.Vector3dVector(points)
50     pcd.colors = o3d.utility.Vector3dVector(colors)
51
52     return pcd # PointCloud with 262144 points.

```

2.

Next, apply a series of coordinate transformations to convert the point clouds from each frame into the world coordinate system. The procedure is as follows:

2.1

For the first frame, we use `gt_poses[0]` to transform its point cloud from the camera coordinate system to the world coordinate system.

```

cam0_rgb = o3d.io.read_image(rgb_files[0])
cam0_depth = o3d.io.read_image(depth_files[0])
cam0_pcd = depth_image_to_point_cloud(cam0_rgb, cam0_depth)

cam0_to_world = gt_poses[0].copy()
pcd0 = deepcopy(cam0_pcd)
pcd0.transform(cam0_to_world)

accumulated_pcd = pcd0
camera_poses = [cam0_to_world] # camera_poses[-1] is the transformation from cam_target to world

```

2.2

For the second frame, use RANSAC and local ICP to estimate a transformation matrix, denoted as `trans_icp`, which aligns the second frame's point cloud to the first frame's coordinate system. Then, by applying `gt_poses[0]`, we can further transform these points into the world coordinate system.

In this way, both frames are represented in the same world coordinate system. The transformation `gt_poses[0]@trans_icp` therefore maps the second frame's point cloud into the world coordinate system.

2.3

For subsequent frames, follow the same procedure: using ICP to estimate the transformation that aligns the current frame(source) to the previous frame(target), and then chaining transformations to map the point cloud into the world coordinate system.

```

190     for i in range(1, len(rgb_files)):
193
194         # Implement the full pipeline:
195         source_rgb = o3d.io.read_image(rgb_files[i])
196         source_depth = o3d.io.read_image(depth_files[i])
197         target_rgb = o3d.io.read_image(rgb_files[i-1])
198         target_depth = o3d.io.read_image(depth_files[i-1])
199
200         # 1. Convert RGB-D to PointCloud (Task 1)
201         source_pcd = depth_image_to_point_cloud(source_rgb, source_depth) # (262144, 3)
202         target_pcd = depth_image_to_point_cloud(target_rgb, target_depth)
203
204         # 2. Preprocess (Voxel/FPFH/Normals)
205         source_down, source_fpfh = preprocess_point_cloud(source_pcd, voxel_size)
206         target_down, target_fpfh = preprocess_point_cloud(target_pcd, voxel_size)
207
208         # 3. Execute Global Registration (RANSAC)
209         trans_init = execute_global_registration(
210             source_down, target_down,
211             source_fpfh, target_fpfh,
212             voxel_size
213         )
214
215         # 4. Execute Local Registration (ICP - Task 2)
216         trans_icp = local_icp_algorithm(source_down, target_down, trans_init, voxel_size * 1.5)
217
218         # 5. Update camera poses and accumulate points
219         cam_source_to_world = camera_poses[-1] @ trans_icp
220         camera_poses.append(cam_source_to_world)
221
222         source_pcd_world = deepcopy(source_pcd)
223         source_pcd_world.transform(cam_source_to_world)
224         accumulated_pcd += source_pcd_world

```

2.4

After collecting all the point clouds, remove the ceiling (the ones with $y \geq 0.5\text{m}$) so that we can see the camera trajectories from the top after visualization.

```

228     # Post-processing: remove the ceiling [cite: 37]
229     points = np.asarray(accumulated_pcd.points)
230     y = points[:, 1]
231     print(f"y min: {y.min():.4f}, y max: {y.max():.4f}, range: {y.max() - y.min():.4f}")
232
233     mask = points[:, 1] < 0.5 # 假設 0.5 公尺以上是天花板
234     accumulated_pcd = accumulated_pcd.select_by_index(np.where(mask)[0])

```

3.

More details about preprocessing, RANSAC and local ICP.

3.1

To reduce computational cost, voxel downsampling is applied to decrease the number of points and adjust the spatial resolution.

3.2

Since we adopt a point-to-plane method for registration, normal vectors are estimated for the point clouds.

3.3

Before applying local ICP, global registration is performed using RANSAC, which requires the computation of FPFH features.

```
54 def preprocess_point_cloud(pcd, voxel_size):
55
56     """
57     Pre-processing: Voxelization and Normal Estimation [cite: 17, 29]
58     """
59
60     pcd_down = pcd.voxel_down_sample(voxel_size) # built-in function
61
62     # Estimate normals for pcd_down (required for Point-to-Plane ICP)
63     radius_normal = voxel_size * 2.0
64     pcd_down.estimate_normals(
65         o3d.geometry.KDTreeSearchParamHybrid(radius=radius_normal, max_nn=30) # k-dimensional tree
66     )
67
68     # Compute FPFH features for Global Registration [cite: 30] Fast Point Feature Histograms
69     radius_feature = voxel_size * 5.0
70     pcd_fpfh = o3d.pipelines.registration.compute_fpfh_feature( # Feature class with dimension = 33
71         pcd_down,
72         o3d.geometry.KDTreeSearchParamHybrid(radius=radius_feature, max_nn=100)
73     )
74     return pcd_down, pcd_fpfh
```

3.4

Use open 3D built in function `registration_ransac_based_on_feature_matching` to do global registration based on FPFH feature matching.

The `distance_threshold` defines the maximum allowable distance between corresponding points during matching.

`ransac_n` is the number of point correspondences sampled in each RANSAC iteration to estimate a candidate transformation.

```

76 def execute_global_registration(source_down, target_down, source_fpfh, target_fpfh, voxel_size):
77
78     distance_threshold = voxel_size * 1.5
79
80     result = o3d.pipelines.registration.registration_ransac_based_on_feature_matching(
81         source_down, target_down,
82         source_fpfh, target_fpfh,
83         mutual_filter=True,
84         max_correspondence_distance=distance_threshold,
85         estimation_method=o3d.pipelines.registration.TransformationEstimationPointToPoint(),
86         ransac_n=4,
87         checkers=[
88             o3d.pipelines.registration.CorrespondenceCheckerBasedOnEdgeLength(0.9),
89             o3d.pipelines.registration.CorrespondenceCheckerBasedOnDistance(distance_threshold)
90         ],
91         criteria=o3d.pipelines.registration.RANSACConvergenceCriteria(4000, 0.999)
92     )
93     return result.transformation

```

3.5

Use open 3D built in function `registration_icp` to do local registration, refining the alignment between the source and target point clouds.

`threshold` defines the maximum correspondence distance between points in the source and target point clouds. Only point pairs within this distance are considered as valid correspondences during ICP.

`trans_init` is the initial transformation matrix obtained from global registration (RANSAC). This provides a good starting point for ICP to refine the alignment.

```

95 def local_icp_algorithm(source_down, target_down, trans_init, threshold):
96     """
97     TASK 2: Open3D ICP Implementation (REQUIRED) [cite: 32]
98     """
99     # Use o3d.pipelines.registration.registration_icp
100    # Estimation method should be TransformationEstimationPointToPlane()
101
102    result = o3d.pipelines.registration.registration_icp(
103        source_down,
104        target_down,
105        threshold,
106        trans_init,
107        o3d.pipelines.registration.TransformationEstimationPointToPlane(),
108        o3d.pipelines.registration.ICPConvergenceCriteria(max_iteration=50)
109    )
110    return result.transformation

```

4.

4.1

For visualization, use `draw_geometries` to render the reconstructed scene, including the accumulated point cloud, the ground truth camera trajectory, and the predicted camera trajectory.

```
112 def visualize_and_evaluate(reconstructed_pcd, predicted_cam_poses, gt_poses, args):
113     """
114     TASK 3: Evaluation & Visualization [cite: 19, 35-38]
115     """
116     # 1. Create LineSet for estimated trajectory (Red)
117     # 2. Create LineSet for ground truth trajectory (Black)
118
119     pred_xyz = np.array([p[:3, 3] for p in predicted_cam_poses])
120     gt_xyz = np.array([p[:3, 3] for p in gt_poses])
121
122     # 建立預測軌跡線段 (紅色)
123     points = pred_xyz
124     lines = [[i, i+1] for i in range(len(points)-1)]
125     line_set_pred = o3d.geometry.LineSet()
126     line_set_pred.points = o3d.utility.Vector3dVector(points)
127     line_set_pred.lines = o3d.utility.Vector2iVector(lines)
128     line_set_pred.paint_uniform_color([1, 0, 0]) # Red
129
130     # 建立 GT 軌跡線段 (黑色)
131     points_gt = gt_xyz
132     line_set_gt = o3d.geometry.LineSet()
133     line_set_gt.points = o3d.utility.Vector3dVector(points_gt)
134     line_set_gt.lines = o3d.utility.Vector2iVector(lines)
135     line_set_gt.paint_uniform_color([0, 0, 0]) # Black
136
137     o3d.visualization.draw_geometries(
138         [reconstructed_pcd, line_set_pred, line_set_gt], window_name=f"Floor {args.floor} Reconstruction"
139         # [reconstructed_pcd], window_name=f"Floor {args.floor} Reconstruction"
140     )
```

4.2

Compute the Mean L2 error between the predicted and ground truth camera positions.

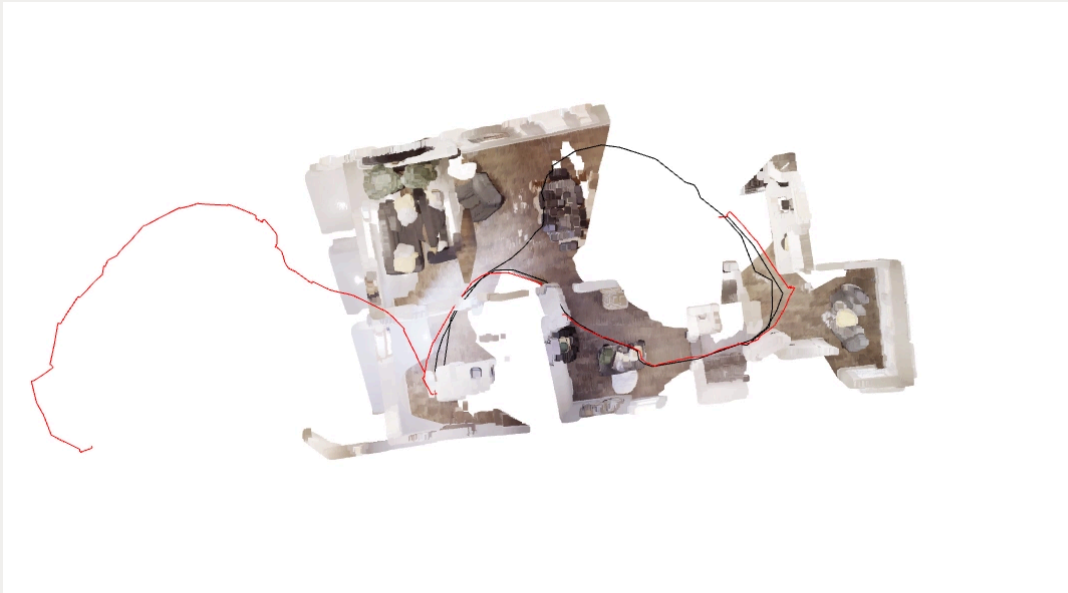
```
143 # Calculate Mean L2 Distance between predicted_cam_poses and gt_poses [cite: 38]
144 #  $L2 = \sqrt{dx^2 + dy^2 + dz^2}$ 
145 mean_l2_error = 0.0
146 errors = []
147 for pred, gt in zip(predicted_cam_poses, gt_poses):
148     pred_t = pred[:3, 3]
149     gt_t = gt[:3, 3]
150     errors.append(np.linalg.norm(pred_t - gt_t))
151
152 mean_l2_error = np.mean(errors)
153 print(f"Mean L2 distance: {mean_l2_error:.6f} meters")
```

Result & Discussion

Screenshots of Results



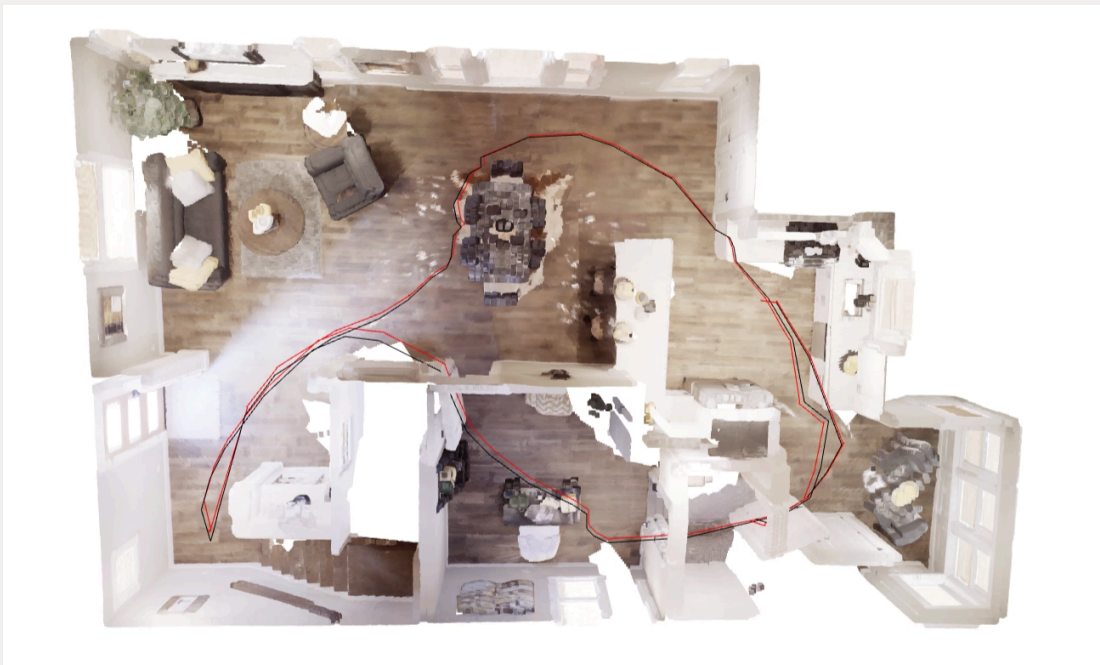
Exp 1



Floor 1 Reconstruction

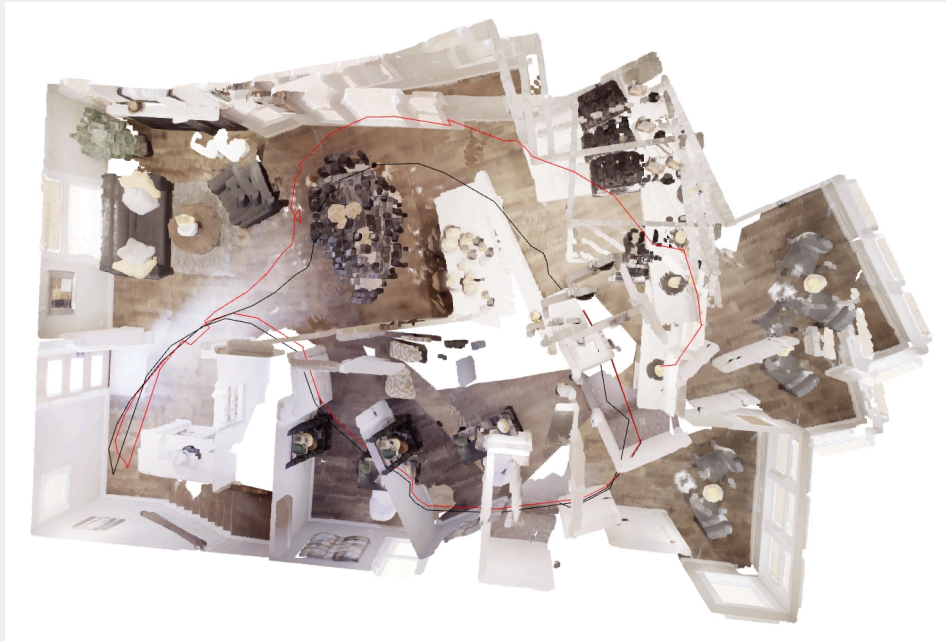


Exp 2



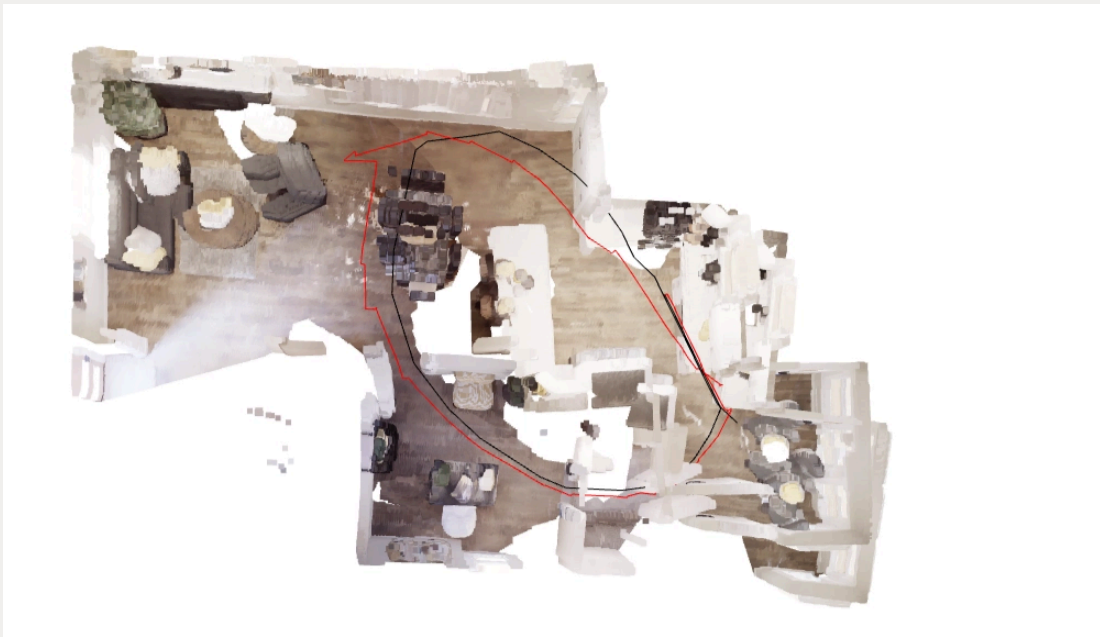


Exp 3

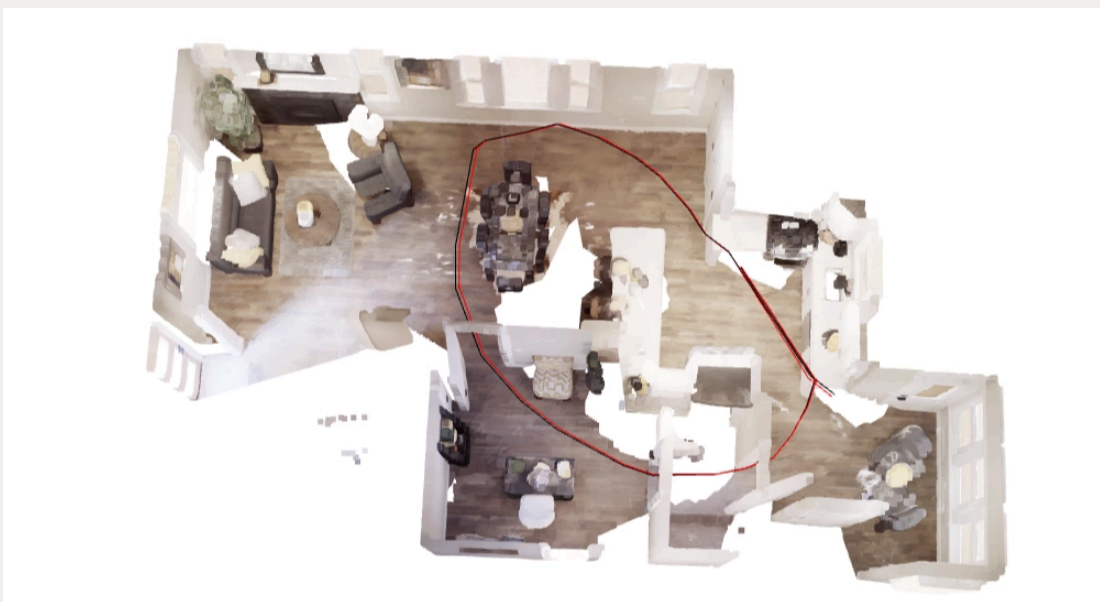




Exp 4

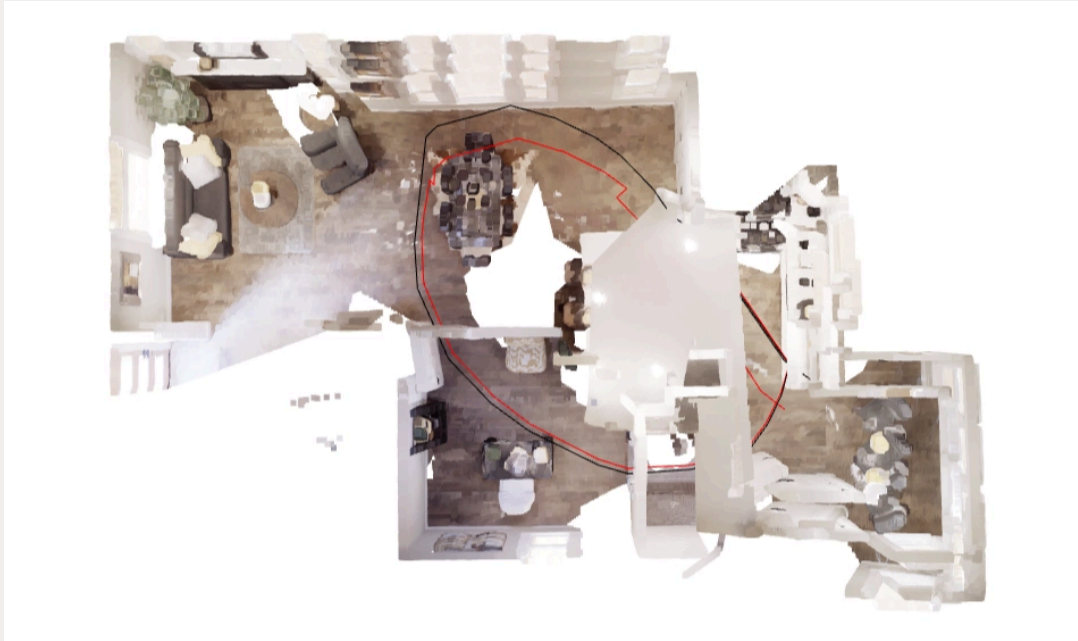


Exp 5





Exp 6



Comparison Table

	data size	voxel size	mean L2 distance(m)	execution time(s)
Exp1	289	0.25	5.66	90.54
Exp2	289	0.08	0.15	153.19
Exp3	289	0.05	0.81	301.21
Exp4	161	0.05	0.55	50.35
Exp5	161	0.08	0.04	87.02
Exp6	161	0.05	0.36	161.73

My Findings

I conducted several experiments to analyze the effects of different configurations. Specifically, we tested two different numbers of frames (161 and 289), and three different voxel sizes (0.25, 0.08, and 0.05), resulting in a total of six experimental settings.

It reveals that a voxel size of 0.08 achieved the best performance.

This can be explained by the trade-off between geometric detail and noise reduction :

When the voxel size is too large (e.g., 0.25), the point cloud becomes overly sparse, and important geometric structures are lost.

When the voxel size is too small (e.g., 0.05), more noise and redundant points are preserved, which can negatively affect feature matching and ICP convergence.

A voxel size of 0.08 provides a good balance between preserving structure and reducing noise

Using 161 frames resulted in a lower mean L2 error compared to using 289 frames.

The result that 160 frames outperform 288 frames may be due to error accumulation.

As more frames are processed, small alignment errors from ICP can accumulate over time, leading to drift in the estimated trajectory.

Questions

1. **What happens if you perform ICP without Global Registration (RANSAC)? Why?**

If we perform ICP without RANSAC, the alignment may fail if the initial poses of the source and target point clouds are far away from each other.

Because ICP is a local optimization method that relies on good initial alignment. If we don't perform RANSAC beforehand, ICP may converge to a wrong local minimum or fail to converge.

Below is an experiment I had tried.



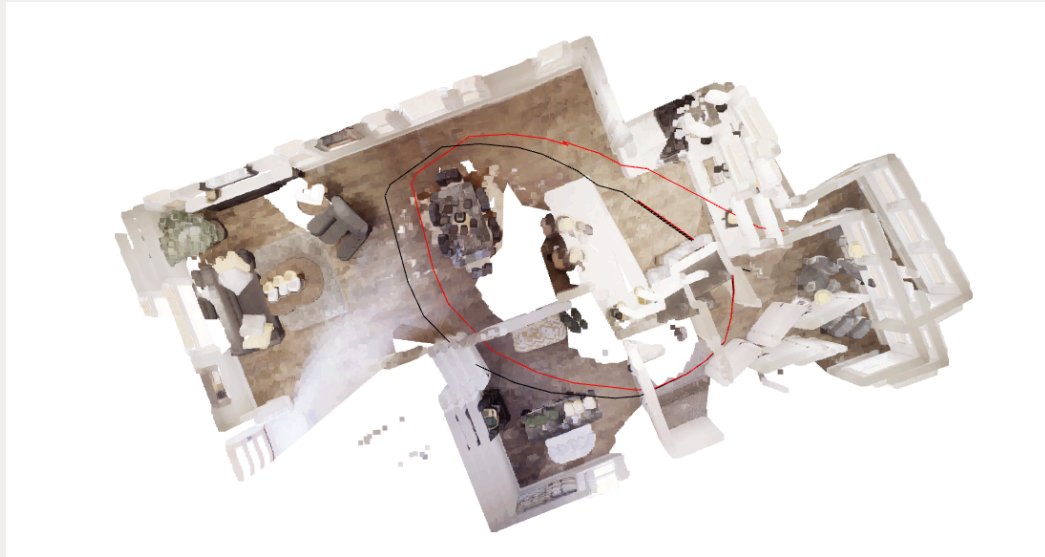
without RANSAC

datasize: 161

voxel size: 0.08

Total execution time: 67.83s

Mean L2 distance: 0.437977 meters





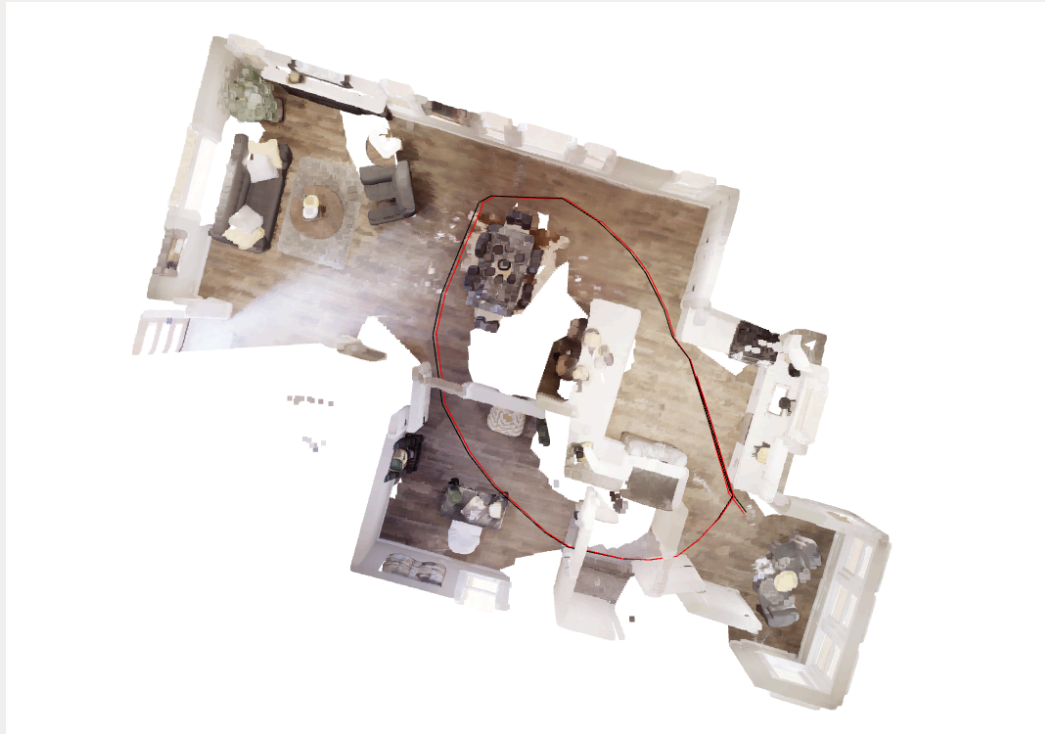
with RANSAC

datasize: 161

voxel size: 0.08

Total execution time: 86.81s

Mean L2 distance: 0.044019 meters



2. Describe any tricks used to improve your ICP stability (e.g., Voxel size, Huber loss, etc.).

Voxel downsampling: Reduces the number of points and suppresses noise, which helps improve both efficiency and robustness.

Proper voxel size selection: A moderate voxel size (e.g., 0.08) provides a good balance between detail and noise.

Global registration (RANSAC): Provides a good initial alignment to avoid poor local minima.

Limiting correspondence distance (threshold): Prevents incorrect point matches, improving stability.

